

Prueba de Caja Blanca

“Sistema de Gestión de Caja para Tags y Productos”

Integrantes:
Leopoldo Bolaños
Matias Medina
Martin Palacios

Fecha: 2026/06/13

REQ007 (Caja Chica):

Implementar una sección dentro del sistema que permita al administrador registrar de forma controlada el dinero de apertura de la jornada y llevar un reporte digital de los vales de egreso generados por gastos menores.

1. CÓDIGO FUENTE para Caja Chica

Controlador (Código)

```
package espe.edu.ec.programacaja.controller;

import espe.edu.ec.programacaja.model.CashFlowModel;
import espe.edu.ec.programacaja.view.CashFlowPanel;
import java.math.BigDecimal;
import java.util.Locale;
import java.util.regex.Pattern;
import javax.swing.JOptionPane;

public class CashFlowController {
    private CashFlowPanel view;
    private CashFlowModel cashFlowModel
    private static final Pattern MONEY_PATTERN = Pattern.compile("^\\d+(\\.\\d{1,2})?$");
    public CashFlowController(CashFlowPanel view, CashFlowModel cashFlowModel) {
        this.view = view;
        this.cashFlowModel = cashFlowModel
        initEvents();
        actualizarReporte();
    }

    private void initEvents() {
        view.getBtnActualizarReporte().addActionListener(e -> actualizarReporte());
        view.getBtnGuardarCajaChica().addActionListener(e -> guardarCajaChica(true));
        view.getBtnGuardarJornada().addActionListener(e -> guardarJornada());
    }

    private void actualizarReporte() {
        view.setReporteFinal(cashFlowModel.getFinalReport());
    }
    private boolean guardarCajaChica(boolean mostrarMensaje) {
        try {
            if (view.isCajaChicaActualEditable()) {
                double cajaChicaActual = parseMoneyAmount(
                    view.getCajaChicaActualTexto(),
                    "caja chica actual"
                );
                cashFlowModel.setInitialCurrentPettyCashAmount(cajaChicaActual);
                view.setCajaChicaActualTexto(String.format(Locale.US, "%.2f", cajaChicaActual));
                view.setCajaChicaActualEditable(false);
            }
            double cajaChicaManana = parseMoneyAmount(
                view.getCajaChicaMananaTexto(),
                "caja chica para mañana"
            );
            cashFlowModel.setNextDayPettyCashAmount(cajaChicaManana);
        } catch (Exception e) {
            JOptionPane.showMessageDialog(view, e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

```

view.setCajaChicaMananaTexto(String.format(Locale.US, "%.2f", cajaChicaManana));

if (mostrarMensaje) {
    JOptionPane.showMessageDialog(
        view,
        "Caja chica registrada correctamente.",
        "Caja chica",
        JOptionPane.INFORMATION_MESSAGE
    );
}

actualizarReporte();
return true;

} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(
        view,
        ex.getMessage(),
        "Error en caja chica",
        JOptionPane.ERROR_MESSAGE
    );
    return false;
}
}

private double parseMoneyAmount(String text, String fieldName) {
    if (text == null || text.trim().isEmpty()) {
        throw new IllegalArgumentException("El campo " + fieldName + " es obligatorio.");
    }

    String normalizedText = text.trim().replace(",", ".");

    if (!MONEY_PATTERN.matcher(normalizedText).matches()) {
        throw new IllegalArgumentException(
            "El valor de " + fieldName + " debe ser numérico y tener máximo 2 decimales. Ejemplo:
20.00"
        );
    }

    BigDecimal amount = new BigDecimal(normalizedText);

    if (amount.compareTo(BigDecimal.ZERO) < 0) {
        throw new IllegalArgumentException("El valor de " + fieldName + " no puede ser negativo.");
    }

    return amount.setScale(2).doubleValue();
}

private void guardarJornada() {
    if (!guardarCajaChica(false)) {
        return;
    }
}

```

```

    }

    if (!cashFlowModel.hasMovements()) {
        JOptionPane.showMessageDialog(
            view,
            "No existen registros guardados para cerrar la jornada.",
            "Cierre de caja",
            JOptionPane.WARNING_MESSAGE
        );
        actualizarReporte();
        return;
    }

    int opcion = JOptionPane.showConfirmDialog(
        view,
        "¿Está seguro de guardar y cerrar la jornada actual?\n"
        + "Revise que el reporte final y la caja chica sean correctos antes de continuar.",
        "Confirmar cierre de jornada",
        JOptionPane.YES_NO_OPTION,
        JOptionPane.QUESTION_MESSAGE
    );

    if (opcion != JOptionPane.YES_OPTION) {
        return;
    }

    String mensaje = cashFlowModel.saveJournalAndReset();

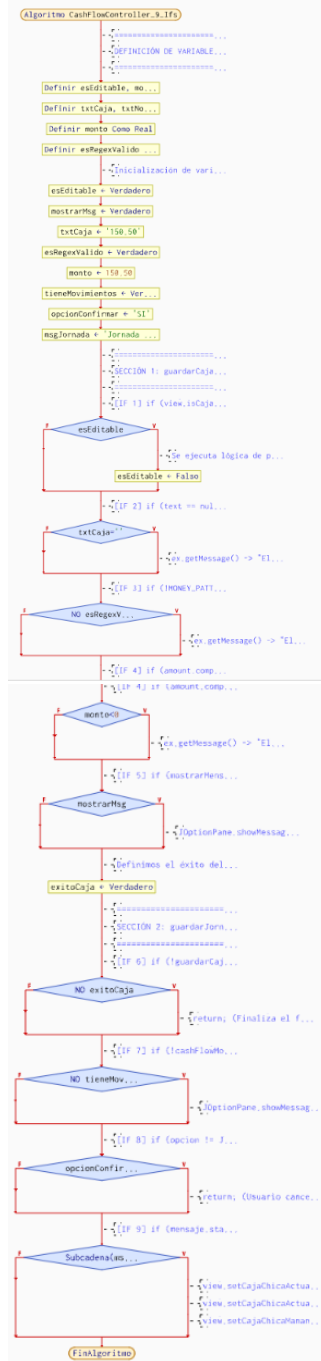
    JOptionPane.showMessageDialog(
        view,
        mensaje,
        "Jornada guardada",
        JOptionPane.INFORMATION_MESSAGE
    );

    if (mensaje.startsWith("Jornada guardada correctamente")) {
        view.setCajaChicaActualTexto(String.format(Locale.US, "%.2f",
        cashFlowModel.getCurrentPettyCashAmount()));
        view.setCajaChicaActualEditable(false);
        view.setCajaChicaMananaTexto("0.00");
    }

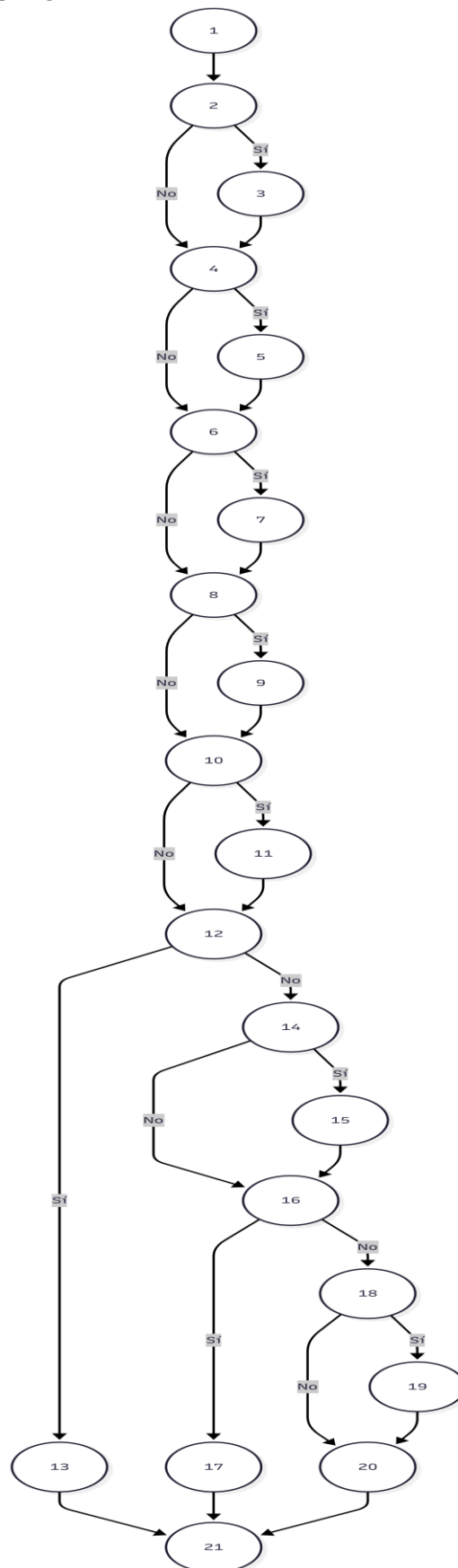
    actualizarReporte();
}
}

```

2. DIAGRAMA DE FLUJO (DF) Caja Chica



3. GRAFO DE FLUJO (GF) Caja Chica



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) Caja Chica

Determinar en base al GF del numeral 4
RUTAS

R1: 1-2-4-6-8-10-12-14-16-18-20-21

R2: 1-2-3-4-6-8-10-12-14-16-18-20

R3: 1-2-4-5-6-8-10-12-14-16-18-20-21

R4: 1-2-4-6-7-8-10-12-14-16-18-20-21

R5: 1-2-4-6-8-9-10-12-14-16-18-20-21

R6: 1-2-4-6-8-10-11-12-14-16-18-20-21

R7: 1-2-4-6-8-10-12-13-21

R8: 1-2-4-6-8-10-12-14-15-16-18-20-21

R9: 1-2-4-6-8-10-12-14-16-17-21

R10: 1-2-4-6-8-10-12-14-16-18-19-20-21

5. COMPLEJIDAD CICLOMÁTICA Caja Chica

Se puede calcular de las siguientes formas:

V/G) Nodos predicados (IF-THEN-ELSE) +1

$V(G) = 9+1$

$V(G) = 10$

$V(G) = A-N + 2$

$V(G) = 29-21+2$

$V(G)=10$

DONDE:

P: Número de nodos predicado

A: Número de aristas

N: Número de nodos